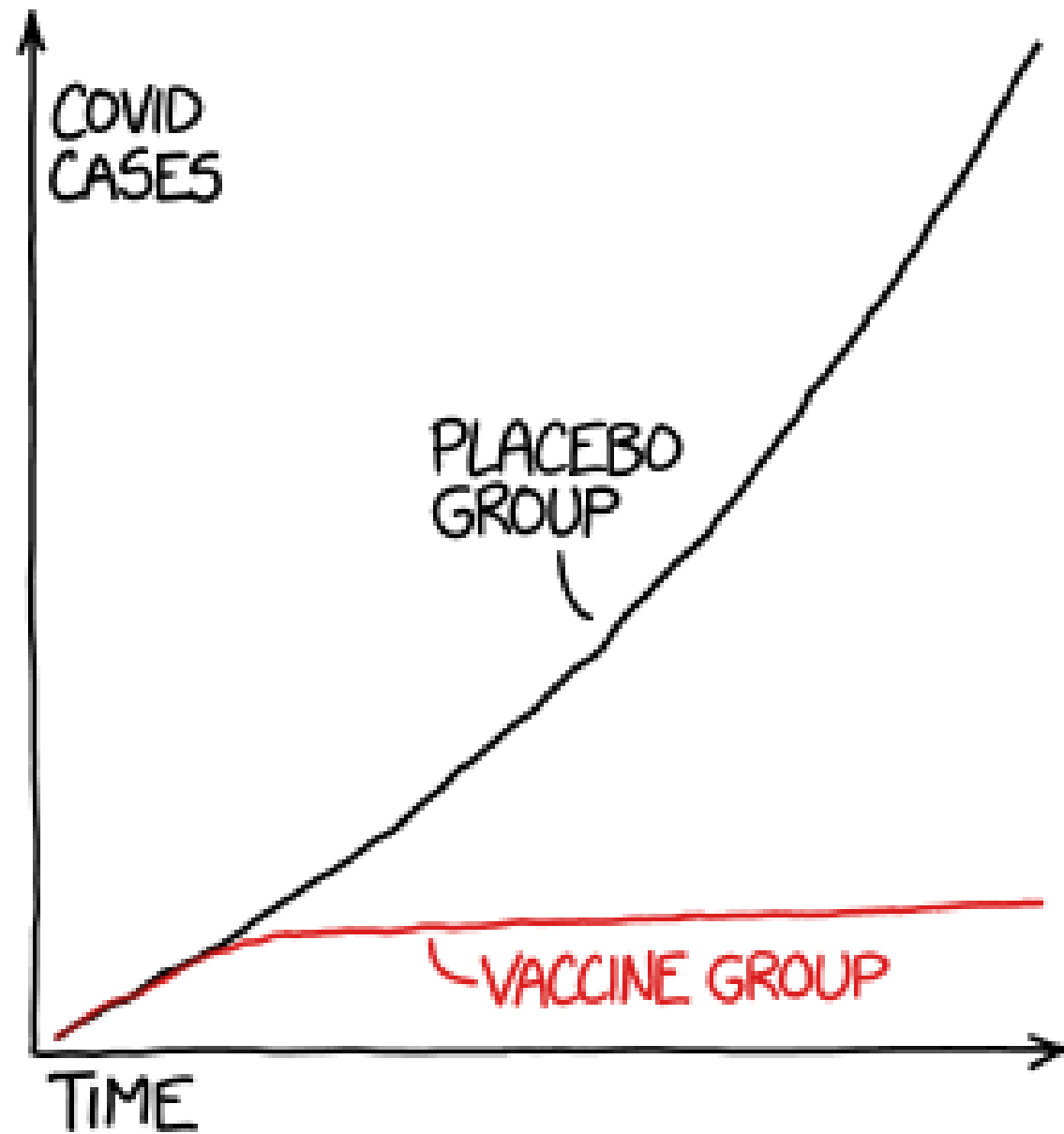




STATISTICS TIP: ALWAYS TRY TO GET DATA THAT'S GOOD ENOUGH THAT YOU DON'T NEED TO DO STATISTICS ON IT

Python for Data Management

Data Science for Economics

Damian Clarke

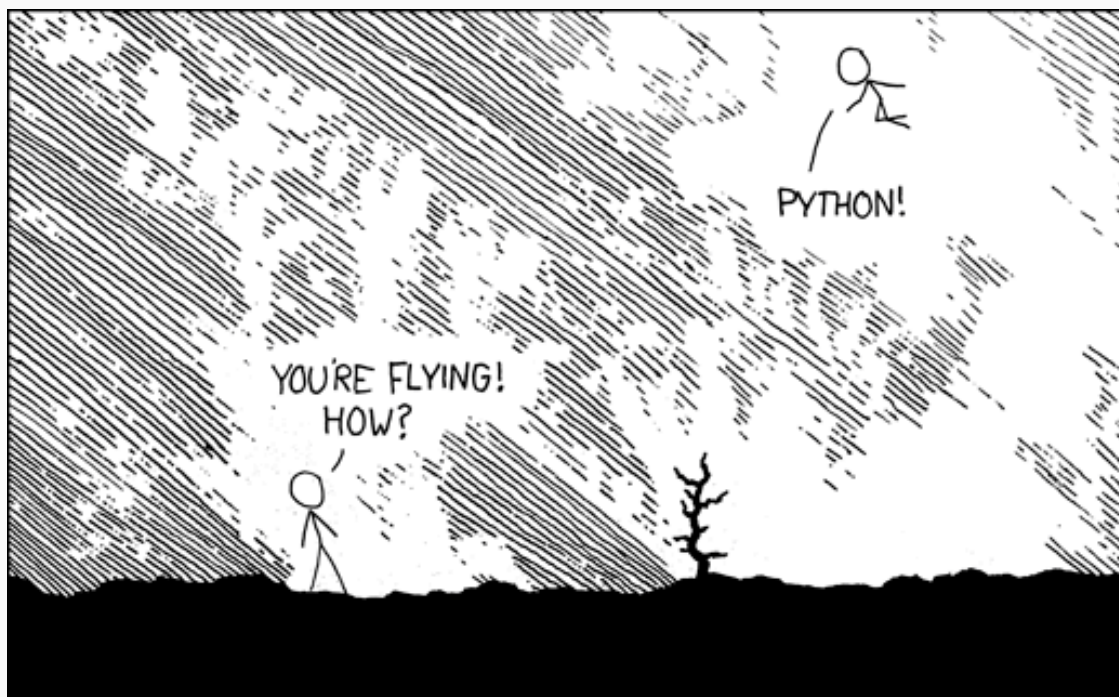
dcc213@exeter.ac.uk

Department of Economics, University of Exeter

26 February, 2026

Getting up and running (again) with Python 🐍

Why Python?



Python is:

- **Readable** - code looks like pseudocode
- **Versatile** - data science, web, automation
- **Popular** - huge community and packages
- **Free** - open source forever

And yes, the flying is pretty cool too! 🚀



Source: xkcd.com/353

A Quick Python Example

```
import numpy as np
import pandas as pd

days = ['Mon', 'Tues', 'Wed', 'Thur', 'Fri']
for i, day in enumerate(days):
    print(day + ', ' + str(i+15) + ' 2024')
```


A Number of Ways to Work

It is important that you have a way to work that is comfortable for you. There are many ways to do this:

1. Stand alone scripts, a text editor, a package installer, and Python at the command line
2. Similar to the above, but with ipython
3. Bundled distribution (Anaconda)
4. Notebooks or interactive computing environments, eg Quarto, Jupyter notebooks
5. Integrated development environments (IDEs, for example VScode, Spyder)

In my own work, I tend to do 1. or 5. In class we will also do 4. I would encourage you to avoid 3, and if you do have Anaconda at present, consider removing, and working out how to do a fresh install.

Where to from here? 🚀

Let's get into it...

Introduction to Pandas 🐼

What is Pandas?

Pandas is Python's primary library for data manipulation and analysis



Not this kind of panda...

- Built on top of NumPy
- Provides two main data structures: **Series** and **DataFrame**
- Designed to make working with “relational” or “labeled” data both easy and intuitive
- Essential for any data science workflow in Python

```
import pandas as pd
import numpy as np
```

Key Data Wrangling Operations

Operation	What it does	Key Pandas tools
Load	Read data from files	<code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_stata()</code>
Inspect	Examine structure and content	<code>.head()</code> , <code>.info()</code> , <code>.describe()</code> , <code>.shape</code>
Select	Choose specific rows/columns	<code>.loc[]</code> , <code>.iloc[]</code> , boolean indexing, <code>.query()</code>
Filter	Keep rows meeting conditions	<code>df[df['col'] > 0]</code> , <code>.query()</code>
Clean	Handle missing/incorrect data	<code>.dropna()</code> , <code>.fillna()</code> , <code>.astype()</code> , <code>.str</code> methods
Transform	Create new variables	<code>df['new'] = ...</code> , <code>.assign()</code> , <code>.apply()</code>
Reshape	Change data layout	<code>.melt()</code> , <code>.pivot()</code> , <code>.pivot_table()</code> , <code>.stack()</code>
Merge	Combine datasets	<code>pd.merge()</code> , <code>pd.concat()</code>
Aggregate	Summarize by groups	<code>.groupby()</code> , <code>.agg()</code> , <code>.transform()</code>
Export	Save results	<code>.to_csv()</code> , <code>.to_excel()</code> , <code>.to_stata()</code>

Why Pandas for Economics?

- Handle time series data (GDP, inflation, stock prices)
- Clean and prepare datasets for econometric analysis
- Merge datasets from different sources (World Bank, FRED, survey data)
- Reshape data between wide and long formats
- Perform group-wise operations and aggregations
- Interface with statistical packages (statsmodels, scikit-learn)

Pandas Data Structures

Series: One-Dimensional Arrays

A Series is a one-dimensional labeled array

```
import pandas as pd
import numpy as np

# Create a Series
gdp_growth = pd.Series([2.3, 2.1, 1.9, -3.4, 5.7],
                        index=[2018, 2019, 2020, 2021, 2022],
                        name='GDP Growth Rate')

print(gdp_growth)
print(f"\nType: {type(gdp_growth)}")
```

2018 2.3

2019 2.1

2020 1.9

2021 -3.4

2022 5.7

Name: GDP Growth Rate, dtype: float64

Type: <class 'pandas.core.series.Series'>

Series Operations

```
# Accessing elements
print(f"GDP growth in 2020: {gdp_growth[2020]}%")
print(f"Average growth: {gdp_growth.mean():.2f}%")

# Boolean indexing
print("\nYears with negative growth:")
print(gdp_growth[gdp_growth < 0])

# Arithmetic operations
print("\nGrowth adjusted for inflation (subtract 2%):")
print(gdp_growth - 2.0)
```

GDP growth in 2020: 1.9%

Average growth: 1.72%

Years with negative growth:

2021 -3.4

Name: GDP Growth Rate, dtype: float64

Growth adjusted for inflation (subtract 2%):

2018 0.3

2019 0.1

2020 -0.1

2021 -5.4

2022 3.7

Name: GDP Growth Rate, dtype: float64

DataFrames: The Workhorse

A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types

```
# Creating a DataFrame
data = {
    'country': ['UK', 'France', 'Germany', 'Italy', 'Spain'],
    'gdp_2022': [3070, 2780, 4080, 2010, 1390], # Billions USD
    'population': [67.0, 65.0, 83.0, 59.0, 47.0], # Millions
    'unemployment': [3.7, 7.3, 3.0, 7.8, 12.9] # Percentage
}

df = pd.DataFrame(data)
print(df)
```

	country	gdp_2022	population	unemployment
0	UK	3070	67.0	3.7
1	France	2780	65.0	7.3
2	Germany	4080	83.0	3.0
3	Italy	2010	59.0	7.8
4	Spain	1390	47.0	12.9

DataFrame Attributes and Methods

```
print("Shape:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nFirst 3 rows:")
print(df.head(3))
```

Shape: (5, 4)

Column names: ['country', 'gdp_2022', 'population', 'unemployment']

Data types:

```
country      object
gdp_2022      int64
population    float64
unemployment  float64
dtype: object
```

First 3 rows:

	country	gdp_2022	population	unemployment
0	UK	3070	67.0	3.7
1	France	2780	65.0	7.3
2	Germany	4080	83.0	3.0

Creating Calculated Columns

```
# GDP per capita
df['gdp_per_capita'] = (df['gdp_2022'] * 1000) / df['population']

# Create categorical variable
df['unemployment_category'] = pd.cut(df['unemployment'],
                                     bins=[0, 5, 10, 15],
                                     labels=['Low', 'Medium', 'High'])

print(df[['country', 'gdp_per_capita', 'unemployment_category']])
```

	country	gdp_per_capita	unemployment_category
0	UK	45820.895522	Low
1	France	42769.230769	Medium
2	Germany	49156.626506	Low
3	Italy	34067.796610	Medium
4	Spain	29574.468085	High

Reading and Writing Data

Reading CSV Files

```
# Basic read
df = pd.read_csv('data.csv')

# With options
df = pd.read_csv('data.csv',
                 sep=',',          # delimiter
                 header=0,         # row for column names
                 index_col=0,      # column to use as index
                 na_values=['NA', ''], # values to treat as NaN
                 parse_dates=['date'], # parse dates
                 dtype={'id': str}) # specify data types
```

Reading from URLs and Other Formats

```
# From URL
url = 'https://raw.githubusercontent.com/...'
df = pd.read_csv(url)

# Excel files
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# JSON
df = pd.read_json('data.json')

# Stata files
df = pd.read_stata('data.dta')
```

Writing Data

```
# CSV
df.to_csv('output.csv', index=False)

# Excel
df.to_excel('output.xlsx', sheet_name='Results', index=False)

# Stata
df.to_stata('output.dta', write_index=False)

# LaTeX table (useful for papers!)
print(df.to_latex(index=False))
```


Data Selection and Indexing

Selecting Columns

```
# Single column (returns Series)
print(df['country'])
print(type(df['country']))

# Multiple columns (returns DataFrame)
print(df[['country', 'gdp_2022']])
```

```
0      UK
1  France
2  Germany
3   Italy
4   Spain
Name: country, dtype: object
<class 'pandas.core.series.Series'>
```

	country	gdp_2022
0	UK	3070
1	France	2780
2	Germany	4080
3	Italy	2010
4	Spain	1390

Selecting Rows: .loc and .iloc

```
# .loc: label-based indexing
print("First row by label:")
print(df.loc[0])

print("\nRows 0 to 2:")
print(df.loc[0:2, ['country', 'gdp_2022']])
```

First row by label:

country	UK
gdp_2022	3070
population	67.0
unemployment	3.7
gdp_per_capita	45820.895522
unemployment_category	Low

Name: 0, dtype: object

Rows 0 to 2:

	country	gdp_2022
0	UK	3070
1	France	2780
2	Germany	4080

.iloc for Position-Based Indexing

```
# .iloc: integer position-based indexing
print("First two rows, first two columns:")
print(df.iloc[0:2, 0:2])

print("\nLast row:")
print(df.iloc[-1])
```

First two rows, first two columns:

	country	gdp_2022
0	UK	3070
1	France	2780

Last row:

country	Spain
gdp_2022	1390
population	47.0
unemployment	12.9
gdp_per_capita	29574.468085
unemployment_category	High

Name: 4, dtype: object

Boolean Indexing: The Key to Filtering

```
# Countries with unemployment > 7%
high_unemp = df[df['unemployment'] > 7]
print(high_unemp[['country', 'unemployment']])

# Multiple conditions (use & for AND, | for OR)
large_low_unemp = df[(df['population'] > 60) & (df['unemployment'] < 5)]
print("\nLarge countries with low unemployment:")
print(large_low_unemp[['country', 'population', 'unemployment']])
```

	country	unemployment
1	France	7.3
3	Italy	7.8
4	Spain	12.9

Large countries with low unemployment:

	country	population	unemployment
0	UK	67.0	3.7
2	Germany	83.0	3.0

Query Method: SQL-like Syntax

```
# Alternative syntax using query()
result = df.query('unemployment > 7 and population > 45')
print(result[['country', 'unemployment', 'population']])

# With variables
threshold = 7
result = df.query('unemployment > @threshold')
print("\nUsing variable:")
print(result[['country', 'unemployment']])
```

	country	unemployment	population
1	France	7.3	65.0
3	Italy	7.8	59.0
4	Spain	12.9	47.0

Using variable:

	country	unemployment
1	France	7.3
3	Italy	7.8
4	Spain	12.9

Data Cleaning and Preparation

The Reality of Data Science



Fun fact: Data scientists spend 80% of their time cleaning data and 20% complaining about cleaning data.

Let's make the 80% less painful! 🧹

Handling Missing Data

```
# Create sample data with missing values
df_missing = pd.DataFrame({
    'country': ['UK', 'France', 'Germany', None, 'Spain'],
    'gdp': [3070, None, 4080, 2010, 1390],
    'year': [2022, 2022, 2022, 2022, None]
})

print("Original data:")
print(df_missing)
print("\nMissing value counts:")
print(df_missing.isnull().sum())
```

Original data:

	country	gdp	year
0	UK	3070.0	2022.0
1	France	NaN	2022.0
2	Germany	4080.0	2022.0
3	None	2010.0	2022.0
4	Spain	1390.0	NaN

Missing value counts:

```
country    1
gdp        1
year       1
dtype: int64
```


Dealing with Missing Data

```
# Drop rows with any missing values
print("Drop any missing:")
print(df_missing.dropna())

# Drop rows only if all values are missing
print("\nDrop if all missing:")
print(df_missing.dropna(how='all'))

# Fill missing values
print("\nFill missing GDP with mean:")
print(df_missing.fillna({'gdp': df_missing['gdp'].mean()}))
```

Drop any missing:

	country	gdp	year
0	UK	3070.0	2022.0
2	Germany	4080.0	2022.0

Drop if all missing:

	country	gdp	year
0	UK	3070.0	2022.0
1	France	NaN	2022.0
2	Germany	4080.0	2022.0
3	None	2010.0	2022.0
4	Spain	1390.0	NaN

Fill missing GDP with mean:

	country	gdp	year
0	UK	3070.0	2022.0
1	France	2137.5	2022.0

Data Types and Conversion

```
# Check data types
print("Current types:")
print(df_missing.dtypes)

# Convert types
df_clean = df_missing.copy()
df_clean['year'] = pd.to_numeric(df_clean['year'], errors='coerce')
print("\nAfter conversion:")
print(df_clean.dtypes)
```

Current types:

country	object
gdp	float64
year	float64

dtype: object

After conversion:

country	object
gdp	float64
year	float64

dtype: object

String Operations

```
# Create sample data
names = pd.Series([' John Smith ', 'JANE DOE', 'bob-jones', 'Mary_Wilson'])

# String methods
print("Original:", names.tolist())
print("Stripped:", names.str.strip().tolist())
print("Lower:", names.str.lower().tolist())
print("Replace:", names.str.replace('-', ' ').tolist())
print("Contains 'Smith':", names.str.contains('Smith').tolist())
```

```
Original: [' John Smith ', 'JANE DOE', 'bob-jones', 'Mary_Wilson']
Stripped: ['John Smith', 'JANE DOE', 'bob-jones', 'Mary_Wilson']
Lower: [' john smith ', 'jane doe', 'bob-jones', 'mary_wilson']
Replace: [' John Smith ', 'JANE DOE', 'bob jones', 'Mary_Wilson']
Contains 'Smith': [True, False, False, False]
```

Reshaping Data: Wide vs Long

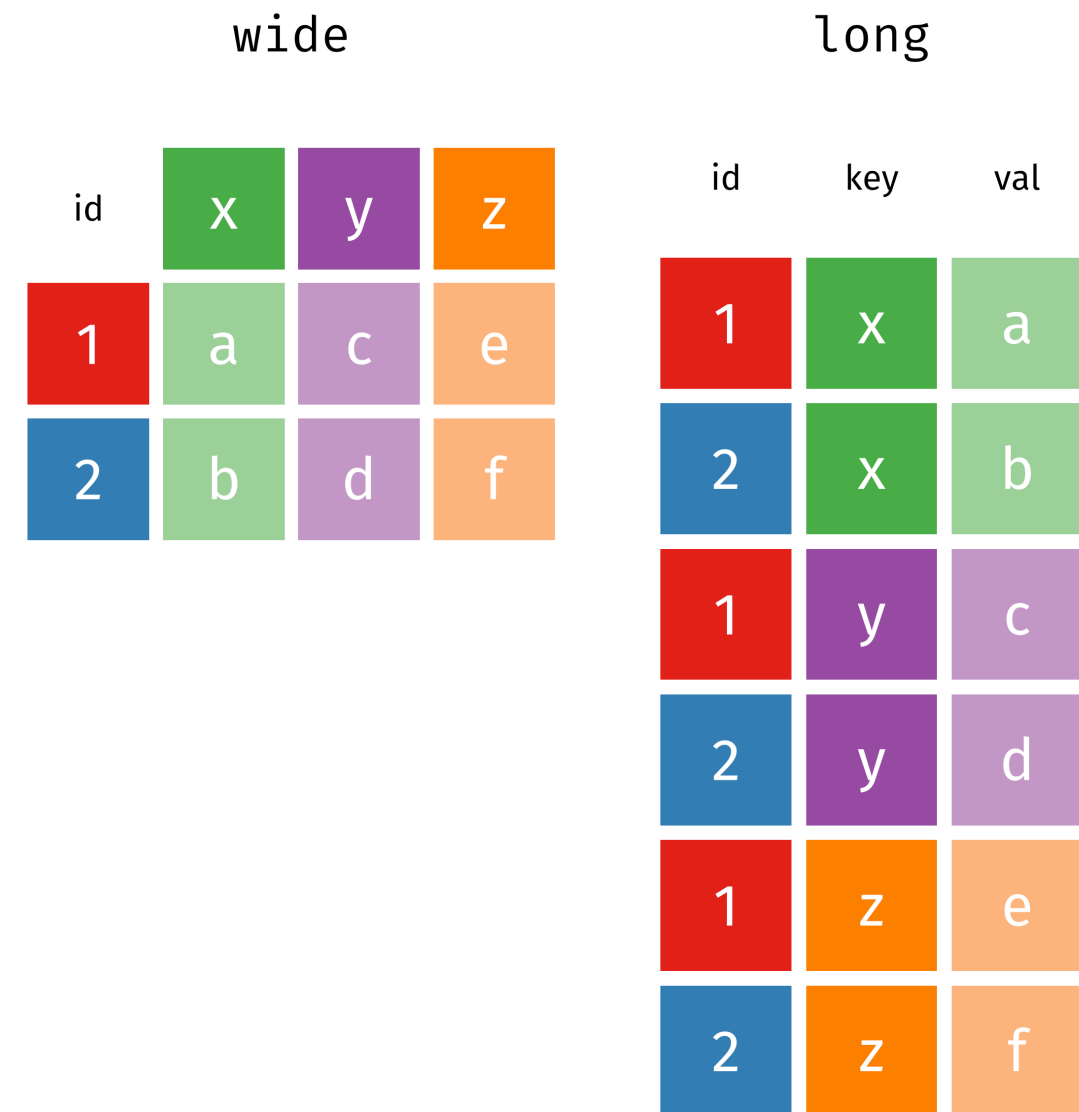
Understanding Wide vs Long Format

Wide format: Each row is an observational unit, variables spread across columns

country	gdp_2020	gdp_2021	gdp_2022
UK	2800	2950	3070
France	2600	2700	2780

Long format: Each row is an observation, with a column indicating the variable

country	year	gdp
UK	2020	2800
UK	2021	2950
UK	2022	3070
France	2020	2600



Wide vs Long (tidy) data (see

<https://github.com/gadenbuie/tidyexplain/>)

Different shapes for different tasks!

Creating Sample Wide Data

```
# Wide format GDP data
gdp_wide = pd.DataFrame({
    'country': ['UK', 'France', 'Germany'],
    'gdp_2020': [2800, 2600, 3750],
    'gdp_2021': [2950, 2700, 3950],
    'gdp_2022': [3070, 2780, 4080]
})

print("Wide format:")
print(gdp_wide)
```

Wide format:

	country	gdp_2020	gdp_2021	gdp_2022
0	UK	2800	2950	3070
1	France	2600	2700	2780
2	Germany	3750	3950	4080

Wide to Long: melt()

```
# Convert to long format
gdp_long = pd.melt(gdp_wide,
                   id_vars=['country'],           # columns to keep
                   value_vars=['gdp_2020', 'gdp_2021', 'gdp_2022'],
                   var_name='year',              # name for variable column
                   value_name='gdp')             # name for value column

print("Long format:")
print(gdp_long)
```

Long format:

	country	year	gdp
0	UK	gdp_2020	2800
1	France	gdp_2020	2600
2	Germany	gdp_2020	3750
3	UK	gdp_2021	2950
4	France	gdp_2021	2700
5	Germany	gdp_2021	3950
6	UK	gdp_2022	3070
7	France	gdp_2022	2780
8	Germany	gdp_2022	4080

Cleaning After Melt

```
# Extract year from variable name
gdp_long['year'] = gdp_long['year'].str.replace('gdp_', '').astype(int)

# Sort by country and year
gdp_long = gdp_long.sort_values(['country', 'year']).reset_index(drop=True)

print("Cleaned long format:")
print(gdp_long)
```

Cleaned long format:

	country	year	gdp
0	France	2020	2600
1	France	2021	2700
2	France	2022	2780
3	Germany	2020	3750
4	Germany	2021	3950
5	Germany	2022	4080
6	UK	2020	2800
7	UK	2021	2950
8	UK	2022	3070

Long to Wide: pivot()

```
# Convert back to wide format
gdp_wide_again = gdp_long.pivot(index='country',
                                columns='year',
                                values='gdp')

print("Back to wide:")
print(gdp_wide_again)

# Reset index to make country a column again
gdp_wide_again = gdp_wide_again.reset_index()
gdp_wide_again.columns.name = None # Remove the columns name
print("\nWith reset index:")
print(gdp_wide_again)
```

Back to wide:

year	2020	2021	2022
country			
France	2600	2700	2780
Germany	3750	3950	4080
UK	2800	2950	3070

With reset index:

	country	2020	2021	2022
0	France	2600	2700	2780
1	Germany	3750	3950	4080
2	UK	2800	2950	3070

Advanced Reshaping: pivot_table()

```
# Sample data with multiple observations per group
sales = pd.DataFrame({
    'region': ['North', 'North', 'South', 'South', 'North', 'South'],
    'product': ['A', 'B', 'A', 'B', 'A', 'B'],
    'quarter': ['Q1', 'Q1', 'Q1', 'Q1', 'Q2', 'Q2'],
    'sales': [100, 150, 120, 140, 110, 145]
})

# Pivot with aggregation
pivot = pd.pivot_table(sales,
                        values='sales',
                        index='region',
                        columns='product',
                        aggfunc='sum',
                        fill_value=0)

print(pivot)
```

	product	A	B
region			
North		210	150
South		120	285

stack() and unstack()

```
# Using our GDP wide data
gdp_indexed = gdp_wide.set_index('country')
print("Original wide:")
print(gdp_indexed.head())

# Stack: columns to rows
gdp_stacked = gdp_indexed.stack()
print("\nStacked:")
print(gdp_stacked.head(6))
```

Original wide:

	gdp_2020	gdp_2021	gdp_2022
country			
UK	2800	2950	3070
France	2600	2700	2780
Germany	3750	3950	4080

Stacked:

country		
UK	gdp_2020	2800
	gdp_2021	2950
	gdp_2022	3070
France	gdp_2020	2600
	gdp_2021	2700
	gdp_2022	2780

dtype: int64

unstack() to Reverse

```
# Unstack: rows to columns
gdp_unstacked = gdp_stacked.unstack()
print("Unstacked:")
print(gdp_unstacked)

# Can also unstack different levels
gdp_stacked.index.names = ['country', 'year']
print("\nStacked with names:")
print(gdp_stacked.head(6))
```

Unstacked:

	gdp_2020	gdp_2021	gdp_2022
country			
UK	2800	2950	3070
France	2600	2700	2780
Germany	3750	3950	4080

Stacked with names:

country	year	
UK	gdp_2020	2800
	gdp_2021	2950
	gdp_2022	3070
France	gdp_2020	2600
	gdp_2021	2700
	gdp_2022	2780

dtype: int64

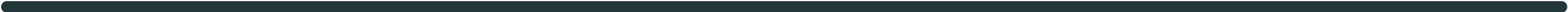
Merging and Joining Data

Types of Merges

Four main types of merges (similar to SQL joins):

- **Inner join:** Only matching rows from both DataFrames
- **Left join:** All rows from left DataFrame, matching from right
- **Right join:** All rows from right DataFrame, matching from left
- **Outer join:** All rows from both DataFrames

Understanding these is **crucial** for combining datasets correctly!



Setup: Two Related Datasets

```
# Economic indicators
econ = pd.DataFrame({
    'country_code': ['GBR', 'FRA', 'DEU', 'ESP'],
    'gdp': [3070, 2780, 4080, 1390],
    'inflation': [2.3, 1.9, 2.1, 3.1]
})

# Population data (note: different countries)
pop = pd.DataFrame({
    'country_code': ['GBR', 'FRA', 'ITA', 'ESP'],
    'population': [67.0, 65.0, 59.0, 47.0],
    'capital': ['London', 'Paris', 'Rome', 'Madrid']
})

print("Economic data:")
print(econ)
print("\nPopulation data:")
print(pop)
```

Economic data:

	country_code	gdp	inflation
0	GBR	3070	2.3
1	FRA	2780	1.9
2	DEU	4080	2.1
3	ESP	1390	3.1

Population data:

	country_code	population	capital
0	GBR	67.0	London
1	FRA	65.0	Paris
2	ITA	59.0	Rome
3	ESP	47.0	Madrid

Inner Join: Only Matches

```
# Inner merge - only countries in both datasets
inner = pd.merge(econ, pop, on='country_code', how='inner')

print("Inner join (GBR, FRA, ESP only):")
print(inner)
print(f"\nRows: {len(inner)} (only complete matches)")
```

Inner join (GBR, FRA, ESP only):

	country_code	gdp	inflation	population	capital
0	GBR	3070	2.3	67.0	London
1	FRA	2780	1.9	65.0	Paris
2	ESP	1390	3.1	47.0	Madrid

Rows: 3 (only complete matches)

Left Join: Keep All Left Records

```
# Left merge - all countries from econ, matching from pop
left = pd.merge(econ, pop, on='country_code', how='left')

print("Left join (all from econ):")
print(left)
print("\nNote: DEU has NaN for population/capital")
```

Left join (all from econ):

	country_code	gdp	inflation	population	capital
0	GBR	3070	2.3	67.0	London
1	FRA	2780	1.9	65.0	Paris
2	DEU	4080	2.1	NaN	NaN
3	ESP	1390	3.1	47.0	Madrid

Note: DEU has NaN for population/capital

Right Join: Keep All Right Records

```
# Right merge - all countries from pop, matching from econ
right = pd.merge(econ, pop, on='country_code', how='right')

print("Right join (all from pop):")
print(right)
print("\nNote: ITA has NaN for gdp/inflation")
```

Right join (all from pop):

	country_code	gdp	inflation	population	capital
0	GBR	3070.0	2.3	67.0	London
1	FRA	2780.0	1.9	65.0	Paris
2	ITA	NaN	NaN	59.0	Rome
3	ESP	1390.0	3.1	47.0	Madrid

Note: ITA has NaN for gdp/inflation

Outer Join: Keep Everything

```
# Outer merge - all countries from both
outer = pd.merge(econ, pop, on='country_code', how='outer')

print("Outer join (all countries):")
print(outer)
print("\nAll countries included, NaN where no match")
```

Outer join (all countries):

	country_code	gdp	inflation	population	capital
0	DEU	4080.0	2.1	NaN	NaN
1	ESP	1390.0	3.1	47.0	Madrid
2	FRA	2780.0	1.9	65.0	Paris
3	GBR	3070.0	2.3	67.0	London
4	ITA	NaN	NaN	59.0	Rome

All countries included, NaN where no match

Merging with Different Column Names

```
# Different column names for the key
pop_renamed = pop.rename(columns={'country_code': 'iso_code'})

# Specify left_on and right_on
merged = pd.merge(econ, pop_renamed,
                  left_on='country_code',
                  right_on='iso_code',
                  how='inner')

print(merged)
print("\nNote: Both key columns retained")
```

	country_code	gdp	inflation	iso_code	population	capital
0	GBR	3070	2.3	GBR	67.0	London
1	FRA	2780	1.9	FRA	65.0	Paris
2	ESP	1390	3.1	ESP	47.0	Madrid

Note: Both key columns retained

Merging on Multiple Columns

```
# Panel data: country-year observations
df1 = pd.DataFrame({
    'country': ['UK', 'UK', 'FR', 'FR'],
    'year': [2021, 2022, 2021, 2022],
    'gdp': [2950, 3070, 2700, 2780]
})

df2 = pd.DataFrame({
    'country': ['UK', 'UK', 'FR'],
    'year': [2021, 2022, 2021],
    'unemployment': [4.5, 3.7, 7.9]
})

# Merge on both country and year
panel = pd.merge(df1, df2, on=['country', 'year'], how='left')
print(panel)
```

	country	year	gdp	unemployment
0	UK	2021	2950	4.5
1	UK	2022	3070	3.7
2	FR	2021	2700	7.9
3	FR	2022	2780	NaN

Validating Merges: The validate Parameter

```
# Ensure one-to-one merge
try:
    pd.merge(econ, pop, on='country_code', validate='1:1')
    print("Validation passed: one-to-one")
except Exception as e:
    print(f"Validation failed: {e}")

# This works
result = pd.merge(econ, pop, on='country_code', validate='1:1', how='inner')
print("\nValidated merge:")
print(result)
```

Validation passed: one-to-one

Validated merge:

	country_code	gdp	inflation	population	capital
0	GBR	3070	2.3	67.0	London
1	FRA	2780	1.9	65.0	Paris
2	ESP	1390	3.1	47.0	Madrid

Merge Validation Options

The `validate` parameter helps catch data quality issues:

- `'1:1'`: One-to-one (each key appears once in each DataFrame)
- `'1:m'`: One-to-many (left keys unique, right keys can repeat)
- `'m:1'`: Many-to-one (left keys can repeat, right keys unique)
- `'m:m'`: Many-to-many (keys can repeat in both)

Best practice: Always use `validate` when you know the expected relationship!

Checking Merge Quality

```
# Check which rows matched
merged = pd.merge(econ, pop, on='country_code', how='outer', indicator=True)
print(merged)
print("\nMerge statistics:")
print(merged['_merge'].value_counts())
```

	country_code	gdp	inflation	population	capital	_merge
0	DEU	4080.0	2.1	NaN	NaN	left_only
1	ESP	1390.0	3.1	47.0	Madrid	both
2	FRA	2780.0	1.9	65.0	Paris	both
3	GBR	3070.0	2.3	67.0	London	both
4	ITA	NaN	NaN	59.0	Rome	right_only

Merge statistics:

```
_merge
both          3
left_only     1
right_only    1
Name: count, dtype: int64
```

Diagnosing Merge Problems

```
# Find non-matching rows
left_only = merged[merged['_merge'] == 'left_only']
print("Only in economic data:")
print(left_only[['country_code', 'gdp']])

right_only = merged[merged['_merge'] == 'right_only']
print("\nOnly in population data:")
print(right_only[['country_code', 'population']])
```

Only in economic data:

	country_code	gdp
0	DEU	4080.0

Only in population data:

	country_code	population
4	ITA	59.0

Common Merge Issues

Problem 1: Unexpected duplicates

```
# Use validate parameter  
pd.merge(df1, df2, on='id', validate='1:1') # Will issue error if duplicates
```

Problem 2: Key formatting differences

```
# Standardize before merging  
df1['id'] = df1['id'].str.strip().str.upper()  
df2['id'] = df2['id'].str.strip().str.upper()
```

Problem 3: Different data types

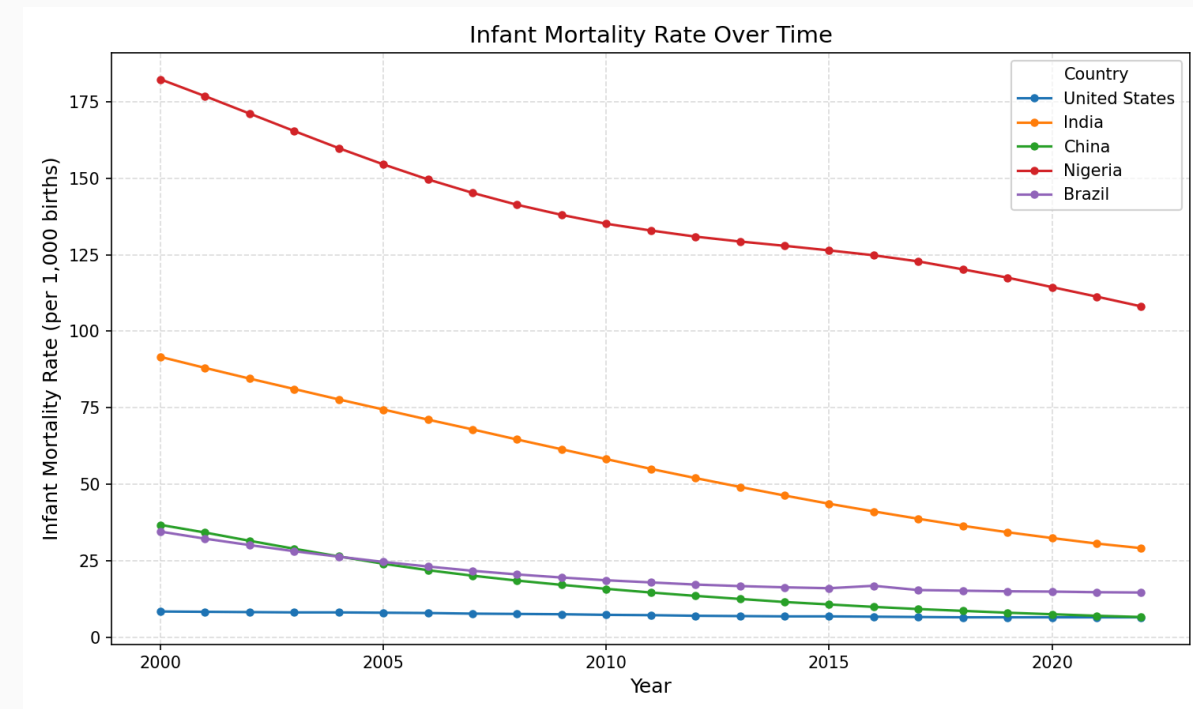
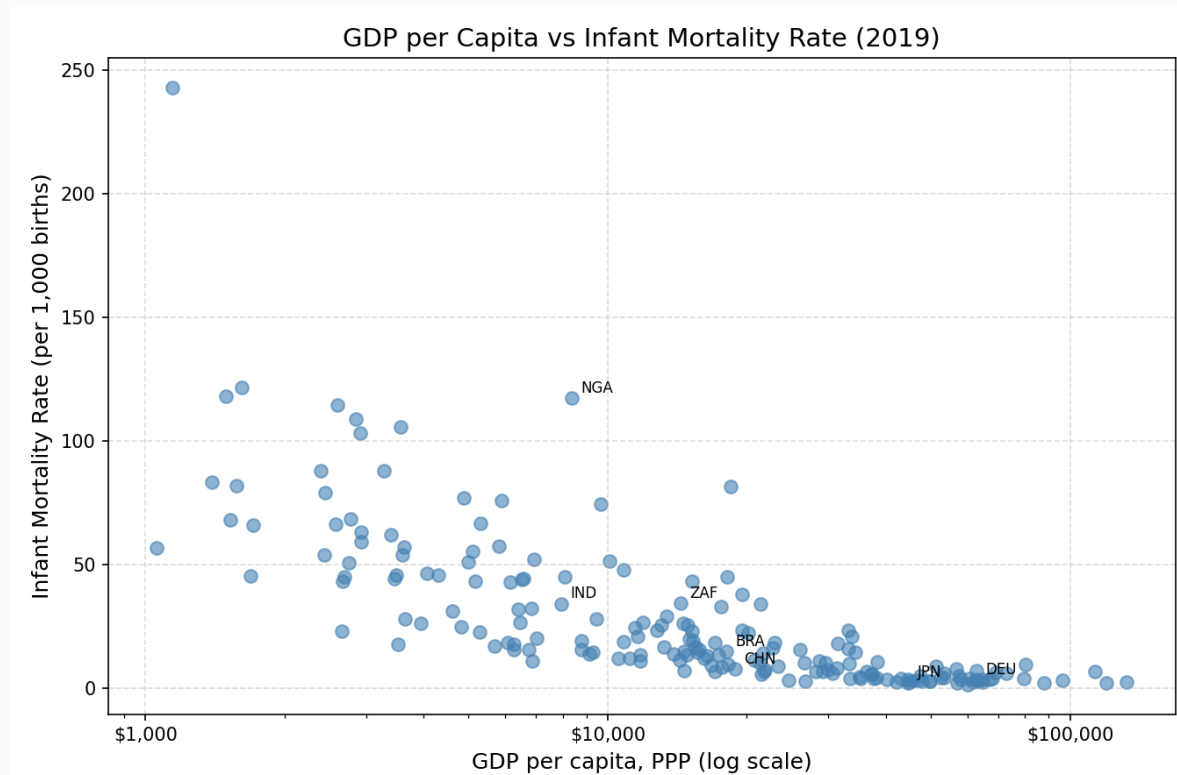
```
# Convert to same type  
df1['year'] = df1['year'].astype(int)  
df2['year'] = df2['year'].astype(int)
```

Always know what your data should look like in theory...

Real-World Example: World Bank Data

I want to look at a real example where we can see some of this in practice.

- I have updated the [github repo](#) with some python code (Just fork and pull)
- This requires one new install `pip install wbgapi`
- We will see that we can use merge and reshape to do build the below plots



Grouping and Aggregation

groupby(): Split-Apply-Combine

```
# Create sample trade data
trade = pd.DataFrame({
    'country': ['UK', 'UK', 'FR', 'FR', 'UK', 'FR'],
    'year': [2021, 2022, 2021, 2022, 2022, 2021],
    'exports': [100, 105, 95, 98, 110, 92],
    'imports': [120, 125, 110, 115, 130, 108]
})

print("Raw data:")
print(trade)
```

Raw data:

	country	year	exports	imports
0	UK	2021	100	120
1	UK	2022	105	125
2	FR	2021	95	110
3	FR	2022	98	115
4	UK	2022	110	130
5	FR	2021	92	108

Basic Aggregation

```
# Group by country and calculate means
by_country = trade.groupby('country')[['exports', 'imports']].mean()
print("Mean by country:")
print(by_country)

# Multiple aggregations
agg_results = trade.groupby('country').agg({
    'exports': ['mean', 'sum', 'std'],
    'imports': ['mean', 'sum']
})
print("\nMultiple aggregations:")
print(agg_results)
```

Mean by country:

	exports	imports
country		
FR	95.0	111.0
UK	105.0	125.0

Multiple aggregations:

	exports			imports	
	mean	sum	std	mean	sum
country					
FR	95.0	285	3.0	111.0	333
UK	105.0	315	5.0	125.0	375

Creating Grouped Variables

```
# Add country-level statistics back to original data
trade['country_avg_exports'] = trade.groupby('country')['exports'].transform('mean')

# Calculate deviation from country mean
trade['exports_deviation'] = trade['exports'] - trade['country_avg_exports']

print(trade)
```

	country	year	exports	imports	country_avg_exports	exports_deviation
0	UK	2021	100	120	105.0	-5.0
1	UK	2022	105	125	105.0	0.0
2	FR	2021	95	110	95.0	0.0
3	FR	2022	98	115	95.0	3.0
4	UK	2022	110	130	105.0	5.0
5	FR	2021	92	108	95.0	-3.0

Working with Time Series

Date Handling

```
# Create date range
dates = pd.date_range('2022-01-01', periods=6, freq='ME')
ts_data = pd.DataFrame({
    'date': dates,
    'value': [100, 102, 105, 103, 108, 110]
})

print(ts_data)
print(f"\nDate type: {ts_data['date'].dtype}")
```

	date	value
0	2022-01-31	100
1	2022-02-28	102
2	2022-03-31	105
3	2022-04-30	103
4	2022-05-31	108
5	2022-06-30	110

Date type: datetime64[ns]

Time Series Operations

```
# Set date as index
ts_data = ts_data.set_index('date')

# Calculate changes
ts_data['pct_change'] = ts_data['value'].pct_change() * 100
ts_data['diff'] = ts_data['value'].diff()

# Rolling statistics
ts_data['rolling_mean_3'] = ts_data['value'].rolling(window=3).mean()

print(ts_data)
```

	value	pct_change	diff	rolling_mean_3
date				
2022-01-31	100	NaN	NaN	NaN
2022-02-28	102	2.000000	2.0	NaN
2022-03-31	105	2.941176	3.0	102.333333
2022-04-30	103	-1.904762	-2.0	103.333333
2022-05-31	108	4.854369	5.0	105.333333
2022-06-30	110	1.851852	2.0	107.000000

Resampling

```
# Create daily data
daily = pd.DataFrame({
    'date': pd.date_range('2022-01-01', periods=90, freq='D'),
    'sales': np.random.randint(50, 150, 90)
})
daily = daily.set_index('date')

# Resample to monthly
monthly = daily.resample('ME').agg({
    'sales': ['sum', 'mean', 'std']
})

print("Monthly aggregation:")
print(monthly.head())
```

Monthly aggregation:

	sales		
	sum	mean	std
date			
2022-01-31	2803	90.419355	30.038613
2022-02-28	2994	106.928571	27.635094
2022-03-31	3383	109.129032	32.273562

Best Practices for Clean Merges

Pre-Merge Checklist

1. Inspect your data

```
df1.info()
df1.head()
df1['key'].value_counts() # Check for duplicates
```

2. Check key columns

```
# Are there duplicates?
print(f"Unique keys in df1: {df1['key'].nunique()}")
print(f"Total rows in df1: {len(df1)}")

# Are there missing values?
print(df1['key'].isna().sum())
```

Pre-Merge Checklist (cont.)

3. Standardize keys

```
# Strip whitespace
df1['key'] = df1['key'].str.strip()

# Consistent case
df1['key'] = df1['key'].str.upper()

# Check data types match
print(df1['key'].dtype)
print(df2['key'].dtype)
```

4. Use validate and indicator

```
result = pd.merge(df1, df2, on='key',
                  how='left',
                  validate='1:1',
                  indicator=True)
```


Post-Merge Validation

```
# Check merge results
print(result['_merge'].value_counts())

# Investigate non-matches
unmatched_left = result[result['_merge'] == 'left_only']
print(f"Unmatched from left: {len(unmatched_left)}")
print(unmatched_left['key'].tolist())

# Check for unexpected NaNs
print("\nMissing values after merge:")
print(result.isna().sum())

# Verify expected row count
expected_rows = ... # depends on merge type
actual_rows = len(result)
assert actual_rows == expected_rows, "Unexpected row count!"
```


Example: Careful Merge Workflow

```
# Load data (simulated)
df1 = pd.DataFrame({
    'country_code': ['GBR', 'FRA', 'DEU', 'ESP'],
    'gdp': [3070, 2780, 4080, 1390]
})

df2 = pd.DataFrame({
    'country_code': ['gbr', 'fra', 'ita', 'esp '], # Note issues!
    'population': [67.0, 65.0, 59.0, 47.0]
})

print("Before cleaning:")
print(df1)
print(df2)
```

Before cleaning:

	country_code	gdp
0	GBR	3070
1	FRA	2780
2	DEU	4080
3	ESP	1390

	country_code	population
0	gbr	67.0
1	fra	65.0
2	ita	59.0
3	esp	47.0

Cleaning and Merging

```
# Standardize keys
df1['country_code'] = df1['country_code'].str.strip().str.upper()
df2['country_code'] = df2['country_code'].str.strip().str.upper()

# Perform merge with validation
result = pd.merge(df1, df2,
                  on='country_code',
                  how='outer',
                  validate='1:1',
                  indicator=True)

print("\nAfter cleaning and merging:")
print(result)
print("\nMerge summary:")
print(result['_merge'].value_counts())
```

After cleaning and merging:

	country_code	gdp	population	_merge
0	DEU	4080.0	NaN	left_only
1	ESP	1390.0	47.0	both
2	FRA	2780.0	65.0	both
3	GBR	3070.0	67.0	both
4	ITA	NaN	59.0	right_only

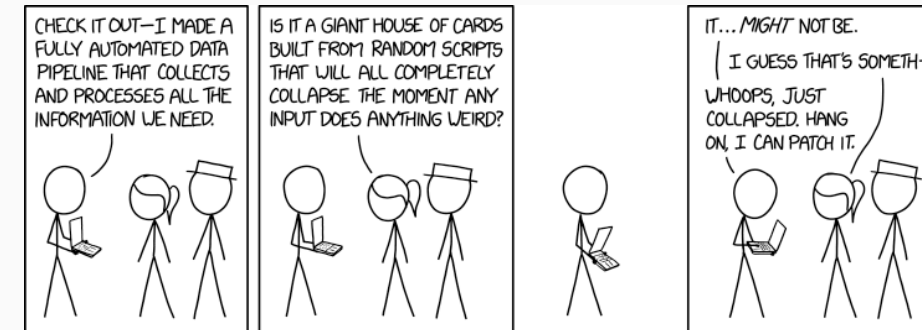
Merge summary:

```
_merge
both          3
left_only     1
right_only    1
Name: count, dtype: int64
```

Summary and Resources

Key Takeaways

1. **DataFrames** are the core data structure - get comfortable with them
2. **Indexing** (`.loc`, `.iloc`, boolean) is essential for data selection
3. **Reshaping** (wide ↔ long) depends on your analysis needs
4. **Merging** requires careful attention to:
 - Key columns (names, types, values)
 - Merge type (inner, left, right, outer)
 - Validation of relationships
 - Post-merge checking



You've got this!

xkcd.com/2054

Essential Methods to Remember

Selection: `.loc[]`, `.iloc[]`, `.query()`

Cleaning: `.dropna()`, `.fillna()`, `.astype()`

Reshaping: `.melt()`, `.pivot()`, `.pivot_table()`

Merging: `pd.merge()` with `validate=` and `indicator=`

Grouping: `.groupby()` with `.agg()` or `.transform()`

Strings: `.str.strip()`, `.str.upper()`, `.str.replace()`

Common Pitfalls to Avoid

- ✗ Not checking for duplicates in merge keys
- ✗ Forgetting to standardize string keys (spaces, case)
- ✗ Mixing data types in merge keys
- ✗ Not using `validate=` parameter
- ✗ Not checking merge results with `indicator=True`
- ✗ Assuming inner join when you need left join
- ✗ Not resetting index after operations
- ✗ Modifying data without creating a copy first

Resources for Further Learning

Official Documentation

- [Pandas User Guide](#)
- [10 Minutes to Pandas](#)

Books

- *Python for Data Analysis* by Wes McKinney (creator of Pandas)
- *Pandas Cookbook* by Theodore Petrou

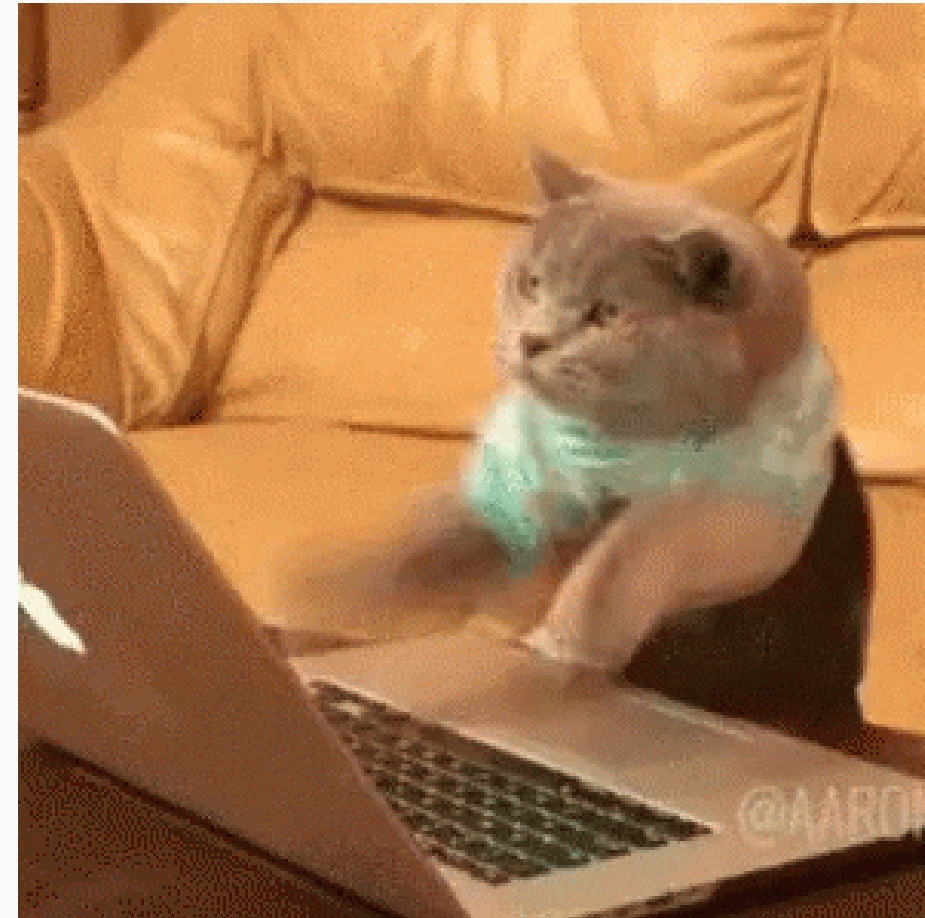
Practice

- Kaggle datasets and tutorials
- Real economic data from FRED, World Bank, Eurostat

Practical Exercise

Using the techniques we've covered:

1. Download two related datasets (e.g., GDP and unemployment by country)
2. Clean and standardize the data
3. Reshape if necessary (wide $\leftrightarrow$ long)
4. Merge the datasets carefully
5. Validate the merge results
6. Perform some grouped analysis
7. Export your results



Time to get your hands dirty with real data!

You're now ready to wrangle data like a pro! 🎉

Remember:

- Start simple, build complexity
- Always validate your merges
- When in doubt, `.head()`
- Manuals, forums, LLMs help, but there is a *big* return to doing this by hand early on!

Pro tip: The best way to learn is to break things and fix them. Don't be afraid to experiment! 🚀

